🔷 Buenas Prácticas MuleSoft

Fuente: Arquitectura de Integración - https://github.com/UisRoPe MuleSoft | Última actualización: Marzo 2026

🤖 GUÍA PARA IA — REGLAS MULESOFT (SÍ / NO HACER)

✅ SÍ HACER — Buenas Prácticas

🏗️ ARQUITECTURA Y FLUJOS

✅ Usar arquitectura de 4 capas: Handler → Orchestrator → Client → Commons

✅ 1 Handler por recurso API (NO 1 handler por operación)

✅ 1 Orchestrator por término de acción (evaluate, retrieve, update, etc.)

✅ 1 Client por recurso API (compartido para todas las operaciones del recurso)

✅ Handler secuencia OBLIGATORIA: Validar headers → Preservar payload → Delegar a orchestrator

✅ Orchestrator secuencia: Dom→Pro (TD_0001) → Validar → Client → Validar → Pro→Dom (TD_0002)

✅ Client: SÓLO HTTP Request, sin transformación ni lógica de negocio

✅ Commons.xml: 6 subflujos reutilizables (validateRequiredHeaders, preserveOriginalPayload, encryptPayloadForPro, validateProResponse, mapProResponseToDom, restoreOriginalContext)

🏠 NAMING CONVENTIONS

✅ Todos los nombres en inglés (kebab-case o lowerCamelCase)

✅ Proyecto: mx-ms-bc-{nombre-api}

✅ Handler: {api}V{version}-handler.xml (ej: apiOperationExampleV1-handler.xml)

✅ Orchestrator: {api}V{version}-{actionTerm}-orchestrator.xml

✅ Client: {api}V{version}-client.xml

✅ DataWeave: TD_0001-Pro_{actionTerm}{apiName}Rq.dwl (Dom→Pro) | TD_0002-Dom_{actionTerm}{apiName}Rs.dwl (Pro→Dom)

✅ Properties: {mule.env}-properties.yaml (dev, qa, prod, local)

✅ Subflows en camelCase SIN guiones (validateRequiredHeaders, NO validate-required-headers)

🔐 SEGURIDAD

✅ Encriptar propiedades sensibles con SecurePropertiesTool (AES/CBC)

✅ Usar HTTPS en todas las comunicaciones externas (TLS Context configurado)

✅ Validar token Bearer en subflujo validate-token.xml (commons)

✅ Headers OBLIGATORIOS: Authorization (Bearer) + X-Consumer-Request-ID (alfanumérico 8-32 chars)

✅ Valores sensibles en pom.xml como <secureProperties> (no visibles en despliegue)

✅ Correlation ID implementado para trazabilidad

✅ Eliminar API Console en producción (puede exponer datos reales)

✅ Aplicar políticas de seguridad en API Manager

📝 DATAWEAVE

✅ Externalizar DataWeave en archivos .dwl (NO inline en XML)

✅ Usar indent=false para respuestas grandes (mejor performance)

✅ Definir Content-Type de ingreso para evitar logs verbosos

✅ Usar streaming=true para payloads grandes (output application/json streaming=true)

✅ Usar variables locales en DWL para no recalcular expresiones

✅ Preferir map sobre reduce (mayor performance)

✅ Validar nulos: usar default y operadores condicionales

🚨 MANEJO DE ERRORES

✅ Implementar 12 handlers APIKIT activos (400, 401, 403, 404, 405, 406, 408, 415, 429, 500, 501 + buildErrorResponse)

✅ Usar namespace CUSTOM para errores propios (type="CUSTOM:UNAUTHORIZED")

✅ Usar on-error-propagate cuando el error debe fluir al cliente

✅ Respuestas de error con estructura: {code, type, message, timestamp}

✅ Error handler a nivel Flow (nunca solo nivel Try)

🧪 MUNIT Y CI/CD

✅ Cobertura mínima del 80% (failBuild:true + requiredApplicationCoverage:80)

✅ Usar dynamicPort https.port (NO http.port) para MUnit

✅ Mockear respuestas de componentes externos en cada test suite

✅ Incluir archivos JSON de ejemplo en src/test/resources para cada operación

✅ Integrar SonarQube plugin 3.11+ para análisis de calidad de código

📊 LOGGING Y MONITOREO

✅ Logs informativos y descriptivos (NO exponer datos sensibles)

✅ Nivel INFO en producción, DEBUG solo cuando sea necesario

✅ Usar Runtime Manager → Deployments → Mule Logs para consultar

✅ Integrar con Datadog (log4j2.xml configurado con ${sys:mule.app.name})

❌ NO HACER — Antipatrones y Errores Críticos

🏗️ ARQUITECTURA Y FLUJOS

❌ NO crear 2 handlers para el mismo recurso API (uno por operación)

❌ NO crear 2 clients para el mismo recurso API

❌ NO poner lógica de negocio en el Handler (solo validar, preservar, delegar)

❌ NO hacer HTTP Request directo desde el Orchestrator (usar Client)

❌ NO poner transformación de datos en el Client

❌ NO poner lógica de negocio en Commons (solo subflujos reutilizables)

❌ NO crear aplicaciones monolíticas (un solo XML enorme)

🏠 NAMING Y CONFIGURACIÓN

❌ NO usar backslash en properties: INCORRECTO properties\${mule.env} (usar forward slash)

❌ NO usar mule.env default="local" en global-config.xml (usar "dev")

❌ NO usar typo base-paht (correcto: base-path)

❌ NO dejar APIKit Config comentado (bloquea validación RAML)

❌ NO usar Maven embebido (configurar Maven externo)

❌ NO sincronizar mal minMuleVersion entre mule-artifact.json y pom.xml

🔐 SEGURIDAD

❌ NO dejar valores JKS en plaintext en properties de qa/prod (encriptar ![EncryptedValue==])

❌ NO exponer datos sensibles en logs

❌ NO mantener API Console activa en producción

❌ NO omitir validación de token Bearer

📝 **DATAWEAVE**

❌ NO escribir DataWeave inline en XML (dificulta mantenimiento y reuso)

❌ NO anidar funciones excesivamente en una sola expresión

❌ NO transformar payload completo si solo se necesitan ciertos campos

❌ NO usar operaciones de lectura repetidas sobre el mismo campo sin variable local

🚨 MANEJO DE ERRORES

❌ NO dejar handlers APIKIT comentados (devuelven 500 genérico en lugar de código correcto)

❌ NO usar estrategia de reconexion "Forever" (causa error recurrente infinito en despliegues)

❌ NO omitir handler para CUSTOM:UNAUTHORIZED (devuelve 500 en lugar de 401)

🧪 MUNIT Y CI/CD

❌ NO usar dynamicPort http.port en MUnit (viola SonarQube)

❌ NO omitir failBuild:true (pipeline pasa con 0% de cobertura)

❌ NO omitir archivos 406.json en recursos de test (NullPointerException)

📊 LOGGING Y MONITOREO

❌ NO aumentar drasticamente logs a nivel INFO (afecta performance)

❌ NO cambiar la traza de logs del Runtime Manager (ajustes en propiedades de despliegue)

❌ NO agregar paquetes org.mule.extension al log forwarding (impacta performance)

❌ NO usar nombre de servicio hardcoded en log4j2.xml (usar ${sys:mule.app.name})

⚙️ CÓDIGO

❌ NO usar código Java personalizado embebido (solo si no hay alternativa en Studio)

❌ NO hardcodear propiedades en XML (externalizar todo en archivos YAML)

🔄 FLUJOS ESTÁNDAR — Patrones Obligatorios

FLUJO HANDLER (entrada)

PATRÓN: HTTP Listener → APIKit Router → [handler flow] → validateRequiredHeaders → preserveOriginalPayload → Flow Ref: [actionTerm]-orchestrator

FLUJO ORCHESTRATOR (lógica)

PATRÓN: DW Transform Dom→Pro (TD_0001) → validateProResponse → Flow Ref: client → Validate response → DW Transform Pro→Dom (TD_0002) → restoreOriginalContext

FLUJO CLIENT (salida)

PATRÓN: HTTP Request (POST) → Headers (Authorization, X-Consumer-Request-ID, Content-Type) → Timeout → Log response

FLUJO ERROR HANDLER (global)

PATRÓN: on-error-propagate type=CUSTOM:UNAUTHORIZED → set httpStatus=401 | on-error-propagate type=ANY → DW build error response → set httpStatus=500

REGLA DE ORO 1:N (Mapeo por Recurso)

1 Recurso API → 1 Handler + N Orchestrators (1 por término de acción) + 1 Client

CORRECTO: /apiOperationExample → apiOperationExampleV1-handler.xml + apiOperationExampleV1-evaluate-orchestrator.xml + apiOperationExampleV1-retrieve-orchestrator.xml + apiOperationExampleV1-client.xml

INCORRECTO: /apiOperationExample → apiOperationExample-evaluate-handler.xml + apiOperationExample-retrieve-handler.xml (MAL: 2 handlers)

2. Mejores Prácticas, Estándares y Lineamientos

2.1 Convención de Nombres

Todos los nombres en inglés. Formatos permitidos:

- Kebab case (minúsculas con guiones medios): algo-como-esto

- Lower camel case: algoComoEsto

Estructura del nombre de los proyecto: { {dominio}-{subdominio} | {sistema} }-{capa}-{tipo}

Ejemplo: mx-ms-bc-bus-doc-mgmt-arch-corp-mule

Capas: exp (experiencia) | bus (negocio) | dom (dominio) | sys (sistema)

2.2 Mejores Prácticas del Diseño de APIs

- Fomentar la modularización: No crear definiciones anidadas. Construir servicios simples primero.

- Evitar integraciones punto a punto: Construir servicios reutilizables.

- Usar integraciones síncronas o asíncronas según el caso de uso.

- Síncronas: feedback instantáneo | Asíncronas: mejores con mayores cargas o servicios lentos.

- Usar construcción adecuada: APIs para servicios interactivos, ETL, batch jobs, etc.

2.2.2 Convenciones de Diseño REST

Recursos: usar sustantivos en plural (/users, /orders, /products). Para más de 2 palabras: minúsculas ambas o kebab-case (/line-items)

Tipos de recursos: Colecciones (/users) | Instancias (/users/{id})

Métodos HTTP: GET (obtener) | PUT (actualizar idempotente) | POST (crear) | PATCH (actualizar parcial) | DELETE (eliminar)

Parámetros de consulta: lowerCamelCase. Ej: dateFrom, dateTo. Colecciones separadas por comas: /users?fields=name,lastName

Media Types: petición usa 'Accept' header | respuesta usa 'Content-Type' header

Campos: lowerCamelCase. Fechas/Hora: ISO8601 en UTC (2021-07-01T13:42:21Z)

Paginación: forzar para respuestas largas. Usar limit y offset

2.2.2.1 Códigos de Respuesta HTTP

Categorías: 1xx Informativo | 2xx Satisfactorio | 3xx Redirección | 4xx Error en cliente | 5xx Error en servidor

Códigos estándar:

- 200 OK: GET/POST/PUT/DELETE - La petición fue satisfactoria

- 201 Created: POST - Nuevo recurso generado, retorna URI del nuevo recurso

- 202 Accepted: POST/PUT/DELETE - Petición aceptada pero no completada aún

- 204 No Content: POST/PUT/DELETE - Sin errores, sin contenido de respuesta

- 400 Bad Request: Petición malformada sintácticamente

- 401 Unauthorized: Requiere autenticación

- 404 Not Found: El servidor no encontró nada para responder la petición

- 405 Method Not Allowed: Método HTTP no permitido

- 406 Not Acceptable: Parámetros no aceptables

- 415 Unsupported Media Type: Formato de entidad no admitido

- 500 Internal Server Error: Condición inesperada que impidió cumplir la solicitud

2.3 Mejores Prácticas sobre Desarrollo e Implementación de APIs

2.3.1.6 Estructura de Archivos de los Proyectos

- Separar en un único XML las configuraciones globales: global-config.xml

- Crear un XML por método de cada recurso implementado.

- Crear XMLs separados para estructuras/lógicas comunes.
- Crear paquetes separados por recurso (dw, wsdls, examples, etc).
- Usar archivos de configuración con extensión YAML para mayor claridad.

2.3.2 Convención de Nombres para la Implementación de APIs

- Proyecto Studio: {nombre-api}-impl (Ej: doc-mgmt-arch-corp-mule-api-impl) → configurar en pom.xml

2.3.3 Estandarización de los Flujos de Mule (Flows)

- Crear múltiples archivos XML Mule con distintos flujos. Desmenuzar aplicación monolítica.
- Crear archivos comunes con flujos reutilizables (transformaciones, logueo, manejo de errores).
- Mantener flujos cortos con única responsabilidad principal.
- Separación de intereses: implementación separada de integración, ruteo y configuración.
- Invocaciones externas envueltas en flujos separados de la implementación.
- Extraer código duplicado a flows comunes.
- Evitar configuraciones no parametrizadas: externalizar todas las propiedades.

2.3.4 Archivo de Propiedades por Ambientes

- Un archivo de propiedades por ambiente: {mule.env}-properties.yaml
- Variable de ambiente: mule.env (valores: dev, qa, uat, prod)
- Propiedades a externalizar: nombres de dominio, URLs, puertos, rutas, timeouts, usuarios, credenciales (encriptadas)
- Doc: https://docs.mulesoft.com/mule-runtime/4.3/deploying-to-multiple-environments

2.3.5 Propiedades de Aplicación Seguras

Propiedades sensitivas (credenciales, contraseñas) deben ser encriptadas.

Doc: https://docs.mulesoft.com/mule-runtime/4.3/secure-configuration-properties

2.3.6 DataWeave

- Usar DataWeave para: transformar/enriquecer datos, crear respuestas y errores en APIs.
- Para peticiones grandes: usar directiva indent=false para mejorar performance.
- Definir Content-Type de ingreso para evitar mensajes verbosos en el log.
- Externalizar código DataWeave en archivos *.dwl para XMLs limpios y reutilización.

Doc: https://docs.mulesoft.com/mule-runtime/4.3/dataweave

2.3.7 APIKit y API Console

- Importar RAML desde Design Center genera flujos automáticos con APIKit.
- Usar manejo de excepciones en el flow del APIKit router.
- SE RECOMIENDA ELIMINAR API Console de la aplicación (puede afectar datos reales).
- Si existe un API Portal, API Console es redundante.
- Si no hay políticas aplicadas, API Console puede ser un riesgo.

2.3.8 Caché

- Usar Cache Scope para llamados HTTP/flujos/lógica que produce información que no cambia frecuentemente.
- Definir TTL (time to live) basado en análisis, no arbitrariamente.

Doc: https://docs.mulesoft.com/mule-runtime/4.3/cache-scope

2.3.9 Estrategias de Reconexión (Reconnection Strategies)

- Incluir estrategias de reconexión en conectores que lo permiten: Salesforce connector, HTTP Request.

- NO usar la estrategia 'Forever'. Puede causar error recurrente infinito durante despliegues.

2.3.10 Código Hecho a Medida Java

- NO se recomienda código Java personalizado embebido en aplicaciones Mule.

- Solo usar cuando no hay alternativa viable en Studio (ej: procesar formatos no estandarizados).

2.3.11 Depuración y Resolución de Problemas

- Usar Anypoint Studio Debugger para depuración.

- Sobreescribir variable 'mule.env' para apuntar a distintos ambientes con precaución (puede provocar cambios no deseados).

2.3.12 Logging

2.3.12.1 Nivel de Logueo

- Los logs generan tráfico de red y operaciones I/O. Priorizar calidad sobre cantidad.

- Logueos deben ser informativos, no exponer datos sensibles.

- Para depuración: usar nivel DEBUG (habilitar solo cuando sea necesario).

2.3.12.2 Depuración a Nivel HTTP

Habilitar: org.mule.service.http.impl.service.HttpMessageLogger para ver peticiones/respuestas detalladas.

Doc: https://help.mulesoft.com/s/article/How-to-Enable-HTTP-Wire-Logging

2.3.13 MUnit

MUnit 2.x es el framework de testing para Mule 4.x. Integrado con Anypoint Studio, ejecutable desde Maven para CI.

Con MUnit se puede:

- Simular respuestas de procesadores

- Espiar resultados de procesadores

- Verificar llamadas a procesadores

- Ver cobertura de tests visualmente en Studio

- Generar reportes de cobertura

Principios para crear MUnit suites:

- Usar respuestas mockeadas para cada componente que llame al exterior.

- Usar set-message para mockear comportamiento con query params, URI params o headers.

- Usar DataWeave para simular peticiones.

Convención de nombres MUnit:

- Test flows: {name-of-the-flow}-unit-test

- Archivos: {name-of-the-mule-file}-unit-test

3. Diseño de APIs y Flujos – Arquetipo

3.1 Plantilla y Composición del Proyecto Arquetipo

Estructura modular: src/main/mule/ contiene:

1. interface(cualquier nombre de preferencia uno relacionado al proyecto).xml - Archivo con el flujo principal (raíz carpeta Mule)

2. client/ - Paquete con los request a consumir externamente (salidas hacia otros servicios)

3. common/ - Elementos reutilizables en el proyecto: configuraciones globales, global-config.xml, validate-token.xml, error-handler.xml

4. handler/ - Agrupación de archivos que sintetizan validaciones iniciales antes de resolver la lógica del flujo. Ej: commons-error-handler.xml, post-register-legal.xml

5. orchestrator/ - Agrupación de archivos que resuelven la lógica de implementación del proyecto. Puede contener subflows. Ej: patch-update-legal.xml, post-register-legal.xml

3.2 Flujo del Handler y Validación de Token

El flujo handler ejecuta secuencialmente:

1. Logger (entrada)

2. Flow Reference: commons-validates-mandatory-consumerRequestId-and-token-header

3. Flow Reference: commons-save-original-payload-and-attributes

4. Flow Reference: {apiOperation}-evaluate-orchestrator

- El header 'bearerToken' debe ser almacenado como variable en el subflujo validate-token.xml

- Actualizar el contrato RAML para que haga referencia a la implementación del proyecto en curso

4. Manejo de Errores

4.1 Jerarquía de Errores en Mule

Los errores Mule tienen espacio de nombres (ej: HTTP, FILE) y un identificador (ej: NOT_FOUND). Jerarquía:

ERRORS → ANY | CRITICAL → SOURCE, TRANSFORMATION, ROUTING, CONNECTIVITY, EXPRESSION, SECURITY, FATAL, OVERLOAD

SECURITY → RETRY_EXHAUSTED, CLIENT_SECURITY, SERVER_SECURITY

La jerarquía permite enrutar errores de forma general a más específica. El orden de On Error es evaluado de arriba a abajo (el primero que coincida se ejecuta).

4.2 Errores Personalizados (Custom Errors)

- Usar namespace CUSTOM para errores propios del proyecto. Ej: type="CUSTOM:UNAUTHORIZED"

- Declarar custom error con componente Raise Error: <raise-error doc:name="TB003" type="CUSTOM:UNAUTHORIZED" />

- Error Mappings: Mapear tipos de error de conectores a custom errors para diferenciar dónde ocurrió el error en el flujo

4.3 Scope Error Handlers y Global Error Handler

Niveles de aplicación del Error Handler:

- Nivel Try: Maneja errores dentro de un bloque Try específico

- Nivel Flow: Maneja errores de todo el flujo

- Global (defaultErrorHandler): Configurado en <configuration defaultErrorHandler-ref="global-error-handler" />

- APIKit Router error handler: Maneja APIKIT:METHOD_NOT_ALLOWED (400), APIKIT:BAD_REQUEST (400), APIKIT:NOT_FOUND (404)

- Usar siempre error-propagate cuando se desea que el error fluya hacia el cliente con su código correspondiente

4.4 Manejo de Logging en Errores

El logging es esencial para monitorear y depurar integraciones. Buenas prácticas:

- Establecer mensajes claros y descriptivos en los logs
- Evitar incluir información sensible en los logs
- Utilizar niveles de log apropiados (INFO en producción, DEBUG solo cuando sea necesario)
- No aumentar drasticamente la cantidad de logs a nivel INFO (puede afectar el performance)
- Monitorear y rotar archivos de logs para evitar problemas de almacenamiento
- Integrar con herramientas externas como Splunk o ELK para análisis avanzado
- El módulo ErrorUtils.dwl provee la función getErrorResponse para construir respuestas de error estandarizadas

5. Seguridad en MuleSoft

MuleSoft ofrece mecanismos nativos para asegurar que solo usuarios y sistemas autorizados accedan a los recursos, protegiendo datos en tránsito y en reposo.

5.2 Encriptación de Propiedades Sensibles

Usar la herramienta SecurePropertiesTool para encriptar valores sensibles en archivos de propiedades:

Comando: java -cp secure-properties-tool.jar com.mulesoft.tools.SecurePropertiesTool string encrypt AES CBC {llave} {cadena_a_encriptar}

- encrypt: Acción (puede ser encrypt o decrypt)
- AES: Algoritmo de encriptación
- CBC: Modo de encriptación
- La llave y los valores encriptados se configuran en secureProperties dentro del pom.xml
- En CloudHub 2.0: Los valores sensibles no deben ser visibles en despliegue, usar etiquetas secureProperties en el pom.xml
- El pipeline de Unity ya incluye capacidad de despliegue con encriptación (no es necesario ajuste manual)

6. Performance y Optimización

6.1 Manejo de Payloads y Streaming

Problemas comunes con uso inadecuado de DataWeave:

- Alto consumo de memoria al procesar payloads grandes
- Latencias elevadas por transformaciones ineficientes
- Cuellos de botella en procesamiento paralelo

Soluciones:

- Habilitar streaming: output application/json streaming=true para JSON/XML
- Filtrar antes de transformar (no transformar datos innecesarios)
- Usar directiva indent=false para respuestas grandes (mejor performance)
- Evitar procesar el payload completo si solo se necesitan ciertos campos

6.2 Optimizaciones Adicionales de DataWeave

- Usar funciones reutilizables en módulos DataWeave (.dwl) importados con 'import'
- Evitar operaciones de lectura repetidas sobre el mismo campo
- Usar variables locales en DataWeave para no recalcular expresiones
- Preferir map sobre reduce cuando sea posible (mayor performance)
- Evitar anidación excesiva de funciones en una sola expresión

7. CI/CD y Versionamiento

9. Monitoreo en Runtime Manager

El monitoreo se realiza desde Anypoint Platform → Runtime Manager.

9.1 Consulta y Uso de Logs

- Acceder a los logs desde Runtime Manager: Aplicación → Deployments → Mule Logs
- Filtrar logs por: identificador, parámetro de búsqueda, tiempo, fecha, período específico
- Filtros de nivel de log disponibles: ERROR, FATAL, INFO, SYSTEM, CONSOLE
- Los logs se visualizan a Nivel INFO por defecto
- Los logs se guardan por 30 días o hasta 100 MB
- Descargar logs para usar en editor de texto para fácil búsqueda

9.2 Buenas Prácticas de Monitoreo

Errores comunes que evitar:

- NO aumentar drasticamente la cantidad de logs a nivel INFO (afecta el performance)
- NO cambiar la traza de logs del Runtime Manager (no modificar ajustes en propiedades de despliegue)
- NO agregar paquetes de org.mule.extension al log forwarding (impacta performance)

Recomendaciones:

- Establecer logs estratégicos para identificar posibles hallazgos
- Usar la Live Console para monitoreo en tiempo real de una aplicación en CloudHub 2.0
- Habilitar log forwarding a Anypoint Platform para centralizar logs (ya activado por defecto en CloudHub)

10. Resumen de Estándares para IA

Esta sección sintetiza los estándares clave para ser usados como contexto por una IA al generar código MuleSoft.

10.1 Nomenclatura (resumen)

- API: {dominio}-{subdominio}-{capa}-api (Ej: customer-academy-experience-api)
- Proyecto Anypoint Studio: {nombre-api}-impl (Ej: orders-proc-api-impl)
- Flows: {verb}-{recurso}-{accion}-flow (snake-case o kebab-case, en inglés)
- Subflows: {verb}-{recurso}-{accion}-subflow
- Archivos XML: {recurso}-{operacion}.xml (Ej: users-get.xml, orders-post.xml)

- Properties: {mule.env}-properties.yaml (Ej: dev-properties.yaml)
- Archivos DWL: {nombre-transformacion}.dwl (en carpeta dw/)

10.2 Estructura de Proyecto Arquetipo (resumen)

```
src/main/
├── mule/
│   ├── interface.xml (flujo principal + APIKit)
│   ├── common/ (global-config.xml, validate-token.xml, error-handler.xml, commons.xml)
│   ├── handler/ (validaciones iniciales por operación)
│   ├── orchestrator/ (lógica de negocio por operación)
│   └── client/ (llamadas a servicios externos)
└── resources/
    ├── dw/ (módulos DataWeave .dwl)
    ├── api/ (RAML y fragmentos)
    └── {mule.env}-properties.yaml
```

10.3 Checklist de Seguridad

- [ ] Propiedades sensibles encriptadas con SecurePropertiesTool (AES/CBC)
- [ ] Validación de token implementada en validate-token.xml
- [ ] HTTPS en todas las comunicaciones externas
- [ ] API Console eliminada en producción

Prerrequisitos del proyecto arquetipo:

- Runtime Mule Server 4.6.7

🏗️ Arquitectura de 4 Capas Obligatoria {#arquitectura}

Diagrama de Flujo

```
          CLIENTE HTTP
               ↓
      ┌──────────────────┐
      │  HTTP Listener   │  ← APIKit Router (valida RAML)
      │  (0.0.0.0:8081)  │
      └──────────────────┘
               ↓
      ┌──────────────────┐
      │    HANDLER       │  ← Capa 1: ENTRADA
      │ (validación)     │  Recibe Dom_Rq (Domain Request)
      └──────────────────┘  Valida headers, preserva payload
               ↓             Delega a orchestrator
      ╔══════════════════╗
      ║   COMMONS        ║     ← Transversal: Subflujos reutilizables
      ║ (subflujos)      ║       Disponibles para todas las capas
      ╚══════════════════╝
               ↑
               ↓
      ┌──────────────────┐
      │  ORCHESTRATOR    │  ← Capa 2: LÓGICA
      │ (transformación) │  Transforma Dom→Pro (DataWeave)
```

```
    └──────────────────┘  Llama Client
              │            Mapea respuesta Pro→Dom
              ↓
    ┌──────────────────┐
    │    CLIENT        │  ← Capa 3: SALIDA
    │ (HTTP Request)   │  HTTP POST a Pro API
    │ (sin lógica)     │  Devuelve response pro
    └──────────────────┘
              │
              ↓
    Sistema Externo (Pro API)
```

Regla De Oro — Mapeo 1:N

```
1 RECURSO API
    ↓
1 Handler (ambas operaciones)
N Orchestrators (1 per action term: evaluate, retrieve, etc.)
1 Client (ambas operaciones)

EJEMPLO CORRECTO ✅
/apiOperationExample →
    apiOperationExample-handler.xml
    apiOperationExample-evaluate-orchestrator.xml
    apiOperationExample-retrieve-orchestrator.xml
    apiOperationExample-client.xml

INCORRECTAMENTE ❌
/apiOperationExample →
    apiOperationExample-evaluate-handler.xml
    apiOperationExample-retrieve-handler.xml            (MAL: 2 handlers)

    apiOperationExample-evaluate-client.xml
    apiOperationExample-retrieve-client.xml             (MAL: 2 clients)
```

Responsabilidades por Capa

Capa	Archivo	Responsabilidad	NUNCA hace
Handler	`{api}-handler.xml`	✅ Recibe solicitud HTTP ✅ Valida headers ✅ Preserva payload original ✅ Delega a orchestrator	❌ Transformaciones de negocio ❌ Lógica de datos ❌ Llamadas HTTP
Orchestrator	`{api}-{term}-orchestrator.xml`	✅ Mapea Dom→Pro (DataWeave) ✅ Llama Client ✅ Mapea Pro→Dom ✅ Validaciones	❌ HTTP directo ❌ Validación de entrada ❌ Error global
Client	`{api}-client.xml`	✅ HTTP Request ✅ Headers ✅ Timeout ✅ Retry ✅ TLS	❌ Transformación de datos ❌ Validación ❌ Lógica negocio
Commons	`commons.xml`	✅ Subflujos reutilizables ✅ Validación de headers ✅ Manejo de encriptación	❌ Lógica de negocio ❌ Integraciones específicas

---

📁 Estructura de Directorios EXACTA {#estructura}

```
mx-ms-bc-{nombre-api}/
│
├── pom.xml ........................... Build + CloudHub 2 + MUnit + SonarQube
├── mule-artifact.json ............... Runtime 4.6.23, Java 17
├── README.md
├── .gitignore
│
└── src/
```

```
├── main/
│   ├── mule/
│   │   ├── {artifactId}-main.xml .......... Entry point + APIKit Router
│   │   │
│   │   ├── client/
│   │   │   └── {apiName}/v1/
│   │   │       └── {apiName}V1-client.xml
│   │   │
│   │   ├── common/
│   │   │   ├── commons.xml ............... 6 subflujos reutilizables
│   │   │   └── global-config.xml ....... Listeners, connections, properties
│   │   │
│   │   ├── handler/
│   │   │   ├── error-handler.xml ....... 12 handlers APIKIT
│   │   │   └── {apiName}/v1/
│   │   │       └── {apiName}V1-handler.xml
│   │   │
│   │   └── orchestrator/
│   │       └── {apiName}/v1/
│   │           ├── {apiName}V1-evaluate-orchestrator.xml
│   │           └── {apiName}V1-retrieve-orchestrator.xml
│   │
│   └── resources/
│       ├── dwl/ ..................... DataWeave transformations
│       │   ├── apiOperation/v1/{action}/
│       │   │   ├── TD_0001-Pro_*Rq.dwl
│       │   │   └── TD_0002-Dom_*Rs.dwl
│       │   ├── commons/ (encriptación, validación)
│       │   └── errorHandler/ (400-501)
│       │
│       ├── examples/ ................ JSON ejemplos (crítico para tests)
│       │   ├── apiOperation/v1/{action}/
│       │   │   ├── Dom_*.json (Domain format)
│       │   │   └── Pro_*.json (Pro API format)
│       │   └── commons/
│       │
│       ├── modules/
│       │   ├── crypto.groovy ........ DESede encryption
│       │   └── ErrorUtils.dwl ...... Error response builder
│       │
│       ├── properties/
│       │   ├── local-properties.yaml (dev plaintext)
│       │   ├── dev-properties.yaml (dev plaintext)
│       │   ├── qa-properties.yaml (encrypted ![...==])
│       │   └── prod-properties.yaml (encrypted ![...==])
│       │
│       ├── log4j2.xml ............... Datadog + archivo
│       └── application-types.xml ... Tipado de datos
│
└── test/
    ├── munit/
    │   ├── main-test-suite.xml
    │   └── apiOperation/v1/
    │       ├── {api}V1-evaluate-test-suite.xml (10 casos)
    │       └── {api}V1-retrieve-test-suite.xml (10 casos)
    │
    └── resources/
        ├── apiOperation/v1/{action}/
```

```
│   └── JSON test files
└── common/
    ├── requestHeaders/ (con/sin token, etc)
    └── response/ (200, errors 400-501)
```

🚨 17 Errores Críticos del Arquetipo {#errores}

ESTOS ERRORES BLOQUEAN DESPLIEGUE. CORREGIR ANTES DE USAR EL ARQUETIPO.

#	Error	Línea / File	Impacto	Síntoma	Corrección
1	`<n>` en pom.xml	pom.xml L20-30	Artefacto sin nombre	Maven warning	Cambiar a `<name>` (3
2	`properties\\${mule.env}`	global-config.xml	FileNotFoundException	Error HTTP 500 en CloudHub	`properties/${mule.e`
3	`base-paht` typo	properties YAML	HTTP Listener en ruta errónea	404 en entrada	`base-path`
4	`dynamicPort http.port`	pom.xml MUnit	SonarQube violation	Quality gate falla	`dynamicPort https.p`
5	Sin `failBuild` en MUnit	pom.xml	Pipeline pasa con 0% cobertura	Baja calidad	Agregar `<failBuild>` `80`
6	`-ecrypt.dwl` (falta 't')	DWL naming	FileNotFoundException	Error HTTP 500	Renombrar a `-encryp`
7	`encrypData-*` (falta 't')	DWL naming	FileNotFoundException	Error HTTP 500	Renombrar a `encrypt`
8	JKS plaintext qa/prod	properties	**VULNERABILIDAD CRÍTICA**	Clave expuesta en repo	`![EncryptedValue==]`
9	APIKIT config comentado	main.xml	Router no valida RAML	Errores APIKIT devuelven 500 genérico	Descomentar `<apiki`
10	APIKIT handlers comentados	error-handler.xml	Sin mapeo de errores RAML	Errores HTTP devuelven 500	Descomentar 12 hand
11	`mule.env` default `local`	global-config.xml	CloudHub carga local-props	Conexiones fallidas en Cloud	Cambiar default a `="`
12	`CUSTOM:UNAUTHORIZED` sin handler	error-handler.xml	Token inválido devuelve 500	Security issue	Agregar handler → 4
13	Service hardcoded en log4j2	log4j2.xml	Todos los APIs igual en Datadog	Imposible trazar	`${sys:mule.app.name`
14	`406.json` faltante	test/resources	NullPointerException test 406	Test suite falla	Crear archivo 406.jso
15	mule-artifact.json desincronizado	mule-artifact.json	Warning runtime version conflict	Incompatibilidad versiones	Sincronizar `minMuleV`
16	SonarQube plugin ausente	pom.xml	No hay análisis de código	Quality gate desactivado	Agregar `sonar-maven`
17	Sin `encrypt.desede.algorithm`	properties	Encriptación sin configurar	Error en tiempo de encriptación	`DESede/CBC/PKCS5Pad`

🔧 pom.xml — Build Correcto {#pom}

Variables a reemplazar: `{nombre-api}`, `{organization-id}`

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xs
d/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
```

```
    <!-- IDENTIDAD DEL PROYECTO -->
    <groupId>834f46f4-ef08-4c50-81ae-56f917b413cc</groupId>
    <artifactId>mx-ms-bc-{nombre-api}</artifactId>
    <version>1.0.0</version>
    <packaging>mule-application</packaging>

    <!-- ✅ CORRECTO: <name> no <n> -->
    <name>mx-ms-bc-{nombre-api}</name>
    <description>API {nombre-api} | CloudHub 2 | Mule 4.6.23</description>

    <!-- PROPIEDADES GLOBALES -->
    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>

        <mule.version>4.6.23</mule.version>
        <mule.maven.plugin.version>4.3.0</mule.maven.plugin.version>
        <munit.version>3.4.0</munit.version>
        <java.version>17</java.version>

        <!-- SonarQube OBLIGATORIO -->
        <sonar.projectKey>mx-ms-bc-{nombre-api}</sonar.projectKey>
        <sonar.projectName>mx-ms-bc-{nombre-api}</sonar.projectName>
        <sonar.language>mule4</sonar.language>
        <sonar.mule.coverage.report.jsonPath>${project.build.directory}/site/munit/coverag
e/munit-coverage.json</sonar.mule.coverage.report.jsonPath>
    </properties>

    <!-- BUILD PLUGINS -->
    <build>
        <plugins>
            <!-- Mule Maven Plugin para CloudHub 2 -->
            <plugin>
                <groupId>org.mule.tools.maven</groupId>
                <artifactId>mule-maven-plugin</artifactId>
                <version>${mule.maven.plugin.version}</version>
                <extensions>true</extensions>
                <configuration>
                    <classifier>mule-application</classifier>
                    <cloudhub2Deployment>
                        <uri>https://anypoint.mulesoft.com</uri>
                        <provider>MC</provider>
                        <environment>${environment}</environment>
                        <businessGroupId>${businessGroupId}</businessGroupId>
                        <target>${target}</target>
                        <muleVersion>${mule.version}</muleVersion>

                        <!-- Connected App (seguro, not user/pass) -->
                        <connectedAppClientId>${connectedAppClientId}</connectedAppClientI
d>
                        <connectedAppClientSecret>${connectedAppClientSecret}</connectedAp
pClientSecret>
                        <connectedAppGrantType>client_credentials</connectedAppGrantType>

                        <applicationName>${project.artifactId}</applicationName>
                        <replicas>${replicas}</replicas>
                        <vCores>${vCores}</vCores>
```

```
                        <properties>
                            <mule.env>${mule.env}</mule.env>
                        </properties>

                        <!-- ✅ CRÍTICO: Propiedades sensibles AQUÍ -->
                        <secureProperties>
                            <datadog.api.key>${datadog.api.key}</datadog.api.key>
                            <encrypt.desede.jksBase64>${encrypt.desede.jksBase64}</encryp
t.desede.jksBase64>
                            <encrypt.desede.password>${encrypt.desede.password}</encrypt.d
esede.password>
                            <mule.key>${mule.key}</mule.key>
                        </secureProperties>

                        <releaseChannel>LTS</releaseChannel>
                        <javaVersion>17</javaVersion>
                    </cloudhub2Deployment>
                </configuration>
            </plugin>

            <!-- ✅ MUnit Plugin con SonarQube Integration -->
            <plugin>
                <groupId>com.mulesoft.munit.tools</groupId>
                <artifactId>munit-maven-plugin</artifactId>
                <version>${munit.version}</version>
                <executions>
                    <execution>
                        <id>test</id>
                        <phase>test</phase>
                        <goals>
                            <goal>test</goal>
                            <goal>coverage-report</goal>
                        </goals>
                    </execution>
                </executions>
                <configuration>
                    <coverage>
                        <runCoverage>true</runCoverage>
                        <!-- ✅ CORRECTO: Falla si < 80% -->
                        <failBuild>true</failBuild>
                        <requiredApplicationCoverage>80</requiredApplicationCoverage>
                        <requiredResourceCoverage>80</requiredResourceCoverage>
                        <requiredFlowCoverage>80</requiredFlowCoverage>
                        <formats>
                            <format>html</format>
                            <format>console</format>
                            <format>sonar</format>
                            <format>json</format>
                        </formats>
                    </coverage>
                    <dynamicPorts>
                        <!-- ✅ CORRECTO: https.port NO http.port -->
                        <dynamicPort>https.port</dynamicPort>
                    </dynamicPorts>
                </configuration>
            </plugin>

            <!-- ✅ SonarQube OBLIGATORIO -->
```

```
            <plugin>
                <groupId>org.sonarsource.scanner.maven</groupId>
                <artifactId>sonar-maven-plugin</artifactId>
                <version>3.11.0.3922</version>
            </plugin>
        </plugins>
    </build>

    <!-- DEPENDENCIAS -->
    <dependencies>
        <dependency>
            <groupId>com.mulesoft.munit</groupId>
            <artifactId>munit-runner</artifactId>
            <version>${munit.version}</version>
            <classifier>mule-plugin</classifier>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>com.mulesoft.munit</groupId>
            <artifactId>munit-tools</artifactId>
            <version>${munit.version}</version>
            <classifier>mule-plugin</classifier>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.mule.connectors</groupId>
            <artifactId>mule-http-connector</artifactId>
            <version>1.11.0</version>
            <classifier>mule-plugin</classifier>
        </dependency>
        <dependency>
            <groupId>org.mule.modules</groupId>
            <artifactId>mule-apikit-module</artifactId>
            <version>1.11.10</version>
            <classifier>mule-plugin</classifier>
        </dependency>
        <dependency>
            <groupId>org.mule.modules</groupId>
            <artifactId>mule-scripting-module</artifactId>
            <version>2.1.1</version>
            <classifier>mule-plugin</classifier>
        </dependency>
    </dependencies>

    <!-- REPOSITORIES -->
    <repositories>
        <repository>
            <id>anypoint-exchange-v3</id>
            <name>Anypoint Exchange</name>
            <url>https://maven.anypoint.mulesoft.com/api/v3/maven</url>
        </repository>
    </repositories>

    <distributionManagement>
        <repository>
            <id>anypoint-exchange-v3</id>
            <url>https://maven.anypoint.mulesoft.com/api/v3/organizations/${businessGroupI
d}/maven</url>
```

```
        </repository>
    </distributionManagement>
</project>
```

⚙️ global-config.xml — Configuración Mule {#config-mule}

```
<?xml version="1.0" encoding="UTF-8"?>
<mule xmlns="http://www.mulesoft.org/schema/mule/core"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xmlns:http="http://www.mulesoft.org/schema/mule/http"
      xmlns:secure-properties="http://www.mulesoft.org/schema/mule/secure-properties"
      xsi:schemaLocation="
        http://www.mulesoft.org/schema/mule/core http://www.mulesoft.org/schema/mule/core/
current/mule.xsd
        http://www.mulesoft.org/schema/mule/http http://www.mulesoft.org/schema/mule/http/
current/mule-http.xsd
        http://www.mulesoft.org/schema/mule/secure-properties http://www.mulesoft.org/sche
ma/mule/secure-properties/current/mule-secure-properties.xsd">

    <!-- ✅ CORRECTO: properties/${mule.env} NO properties\\${mule.env} -->
    <configuration-properties file="properties/${mule.env}-properties.yaml"
                              unmarshallType="java.util.HashMap"/>

    <!-- Secure Properties para qa/prod -->
    <secure-properties:config name="Secure_Properties_Config"
                              file="properties/${mule.env}-secure-properties.yaml"
                              key="${mule.key}"
                              algorithm="Blowfish"/>

    <!-- HTTP Listener Configuration -->
    <http:listener-config name="HTTP_Listener_Config" doc:name="HTTP Listener config">
        <http:listener-connection host="0.0.0.0"
                                  port="${http.port:8081}"
                                  protocol="HTTP"/>
    </http:listener-config>

    <!-- HTTP Request Configuration (a Pro API) -->
    <http:request-config name="HTTP_Request_Config"
                         doc:name="HTTP Request config"
                         connectionIdleTimeout="30000"
                         responseTimeout="${pro.api.timeout:10000}">
        <http:request-connection protocol="HTTPS"
                                 host="${pro.api.host}"
                                 port="${pro.api.port:443}"
                                 tlsContext="TLS_Context">
            <http:authentication>
                <http:basic-authentication username="#[p('pro.api.user')]"
                                           password="#[p('pro.api.password')]"/>
            </http:authentication>
        </http:request-connection>
    </http:request-config>

    <!-- TLS Context -->
    <tls:context name="TLS_Context"
                 doc:name="TLS Context"
                 xmlns:tls="http://www.mulesoft.org/schema/mule/tls">
        <tls:key-store path="classpath://client-keystore.jks"
```

```
                        password="${encrypt.desede.password:changeit}"
                        keyPassword="${encrypt.desede.password:changeit}"/>
    </tls:context>

    <!-- ✅ APIKit Config NO COMENTADO -->
    <apikit:config name="APIKit_Config"
                   api="doc:api"
                   outboundHeadersMapName="outboundHeaders"
                   httpStatusVarName="httpStatus"
                   xmlns:apikit="http://www.mulesoft.org/schema/mule/apikit"/>
</mule>
```

📦 commons.xml — 6 Subflujos Obligatorios {#commons}

```
<?xml version="1.0" encoding="UTF-8"?>
<mule xmlns="http://www.mulesoft.org/schema/mule/core"
      xmlns:validation="http://www.mulesoft.org/schema/mule/validation"
      xmlns:ee="http://www.mulesoft.org/schema/mule/ee/core">

    <!-- Subflujo 1: Validar Headers Obligatorios -->
    <sub-flow name="validateRequiredHeaders" doc:name="Validate Required Headers">
        <validation:is-not-empty value="#[attributes.headers.authorization]"
                                 errorMessage="Authorization header required"/>
        <validation:is-not-empty value="#[attributes.headers.'X-Consumer-Request-ID']"
                                 errorMessage="X-Consumer-Request-ID header required"/>
        <validation:matches-regex value="#[attributes.headers.'X-Consumer-Request-ID']"
                                  regex="^[A-Z0-9]{8,32}$"
                                  errorMessage="Invalid X-Consumer-Request-ID format"/>
    </sub-flow>

    <!-- Subflujo 2: Preservar Payload Original -->
    <sub-flow name="preserveOriginalPayload" doc:name="Preserve Original Payload">
        <ee:transform doc:name="Store payload in variable">
            <ee:message>
                <ee:set-variable variableName="originalPayload">#[payload]</ee:set-variabl
e>
            </ee:message>
        </ee:transform>
    </sub-flow>

    <!-- Subflujo 3: Encriptar Payload -->
    <sub-flow name="encryptPayloadForPro" doc:name="Encrypt Payload for Pro">
        <ee:transform doc:name="Prepare encryption parameters">
            <ee:message>
                <ee:set-payload resource="classpath://dwl/commons/encryptData-parameters.d
wl"/>
            </ee:message>
        </ee:transform>
        <ee:transform doc:name="Execute encryption">
            <ee:message>
                <ee:set-payload resource="classpath://dwl/commons/encryptData-encryptedPay
load.dwl"/>
            </ee:message>
        </ee:transform>
    </sub-flow>

    <!-- Subflujo 4: Validar Respuesta Pro -->
```

```
    <sub-flow name="validateProResponse" doc:name="Validate Pro Response">
        <validation:is-not-null value="#[payload]"
                                errorMessage="Empty response from Pro"/>
        <validation:is-json value="#[payload]"
                            errorMessage="Invalid Pro response format (not JSON)"/>
    </sub-flow>

    <!-- Subflujo 5: Mapear Pro > Dom -->
    <sub-flow name="mapProResponseToDom" doc:name="Map Pro Response to Domain">
        <ee:transform doc:name="Transform Pro > Domain">
            <ee:message>
                <ee:set-payload resource="classpath://dwl/apiOperation/{action}/TD_0002-Do
m_{action}Rs.dwl"/>
            </ee:message>
        </ee:transform>
    </sub-flow>

    <!-- Subflujo 6: Restaurar Contexto -->
    <sub-flow name="restoreOriginalContext" doc:name="Restore Original Context">
        <ee:transform doc:name="Clear variables">
            <ee:message>
                <ee:set-variable variableName="originalPayload">#[null]</ee:set-variable>
            </ee:message>
        </ee:transform>
    </sub-flow>

</mule>
```

📝 Convenciones de Nombres NO NEGOCIABLES {#convenciones}

Patrón: `{scope}.{service}.{property}`

Proyecto

```
mx-ms-bc-{nombre-api}
mx = México
ms = MuleSoft
bc = Bancoppel
```

Handlers

```
{api}[V{version}]-handler.xml
✅ apiOperationExampleV1-handler.xml
❌ apiOperationExample-v1-handler.xml (guiones en version)
```

Orchestrators

```
{api}[V{version}]-{actionTerm}-orchestrator.xml
✅ apiOperationExampleV1-evaluate-orchestrator.xml
❌ apiOperationExample-evaluateOrchestrator.xml
```

Clients

```
{api}[V{version}]-client.xml
✅ apiOperationExampleV1-client.xml
❌ apiOperationExample-service-client.xml
```

DataWeave — TD_XXXX

```
TD_{sequence}-{Tipo}_{actionTerm}{apiName}{direction}.dwl

TD_0001 = Domain → Pro (REQUEST)
TD_0002 = Pro → Domain (RESPONSE)

✅ TD_0001-Pro_evaluateApiOperationRq.dwl
✅ TD_0002-Dom_evaluateApiOperationRs.dwl
✅ TD_0001-Pro_evaluateApiOperationRq-encrypt.dwl
```

Properties

```
{environment}.{service}.{property}

✅ pro.api.host
✅ dom.api.timeout
✅ encrypt.desede.password
✅ datadog.api.key

❌ ProApiHost (camelCase sin puntos)
```

Variables en Flows (DWL/XML)

```
camelCase con prefijo

✅ vars.proResponseCode
✅ vars.encryptedPayload
✅ vars.consumerRequestId
```

Sub-flows

```
camelCase sin guiones

✅ validateRequiredHeaders
✅ encryptPayloadForPro
❌ validate-required-headers
```

📦 mule-artifact.json {#mule-artifact}

```
{
  "name": "mx-ms-bc-{nombre-api}",
  "description": "API {nombre-api} | CloudHub 2 | Mule 4.6.23",
  "minMuleVersion": "4.6.23",
  "maxMuleVersion": "4.9.99",
  "requiredProduct": "MULE",
  "bundleDescriptorVersion": "1.0.0",
  "requiredAddons": [],
  "properties": {
    "mule.env": "dev"
  }
}
```

✅ Checklist CORE {#checklist-core}

Antes de empezar desarrollo

- [ ] pom.xml:
  - [ ] `<name>` correcto (NO `<n>`)
  - [ ] `<artifactId>mx-ms-bc-{nombre-api}`
  - [ ] `mule.version 4.6.23`
  - [ ] `munit.version 3.4.0`
  - [ ] Java 17
  - [ ] `failBuild: true` + `requiredApplicationCoverage: 80`
  - [ ] `dynamicPort https.port` (NO http.port)
  - [ ] SonarQube plugin presente
- [ ] mule-artifact.json:
  - [ ] `minMuleVersion: "4.6.23"`
  - [ ] Sincronizado con pom.xml
- [ ] global-config.xml:
  - [ ] `properties/${mule.env}` (NO backslash)
  - [ ] Default mule.env = "dev" (NO "local")
  - [ ] APIKit Config NO comentado
  - [ ] HTTP Listener configurado
  - [ ] HTTP Request pool configurado
  - [ ] TLS Context presente
- [ ] commons.xml:
  - [ ] 6 sub-flows presentes
  - [ ] Sin lógica de negocio
  - [ ] Sin duplicaciones
- [ ] Estructura de directorios:
  - [ ] `src/main/mule/{handler,orchestrator,client}`
  - [ ] `src/main/resources/{dwl,examples,properties}`
  - [ ] `src/test/munit`
- [ ] Properties:
  - [ ] local, dev, qa, prod presentes
  - [ ] qa/prod con valores encrypted ![...==]
  - [ ] `base-path` correcto (NO `base-paht`)
  - [ ] `encrypt.desede.algorithm` = "DESede/CBC/PKCS5Padding"
- [ ] Error handling:
  - [ ] error-handler.xml con 12 handlers APIKIT activos
  - [ ] Sin comentarios bloqueando handlers
- [ ] Naming:
  - [ ] `encrypt.dwl` (NO `ecrypt.dwl`)
  - [ ] `encryptData-*` (NO `encrypData-*`)
  - [ ] Folders versionados (ex: `/v1/`)

- La estructura del proyecto debe seguir el modelo de arquetipo

Estructura por Operación

Para cada endpoint (evaluate, retrieve):

```
1 Handler XML
N Orchestrator XML (1 por término de acción)
1 Client XML (compartido o específico)
```

📥 HANDLER — Capa de Entrada {#handler}

Estructura Obligatoria

```
RECIBE: Domain Request (Dom_*Rq.json)
VALIDA: Headers (Authorization, X-Consumer-Request-ID)
PRESERVA: Payload original
DELEGA: A Orchestrator
```

Ejemplo: apiOperationExample-handler.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<mule xmlns="http://www.mulesoft.org/schema/mule/core"
      xmlns:apikit="http://www.mulesoft.org/schema/mule/apikit"
      xmlns:http="http://www.mulesoft.org/schema/mule/http"
      xmlns:ee="http://www.mulesoft.org/schema/mule/ee/core">

    <!-- ==================== EVALUATE OPERATION ==================== -->
    <flow name="apiOperationExample-evaluate-handler"
          doc:name="apiOperationExample-evaluate-handler">

        <!-- Paso 1: Validar Headers -->
        <flow-ref name="validateRequiredHeaders"
                  doc:name="Validate Headers"/>

        <!-- Paso 2: Preservar Payload Original -->
        <flow-ref name="preserveOriginalPayload"
                  doc:name="Preserve Original Payload"/>

        <!-- Paso 3: Delegar a Orchestrator -->
        <flow-ref name="apiOperationExample-evaluate-orchestrator"
                  doc:name="Call Evaluate Orchestrator"/>

        <!-- Error Handling -->
        <error-handler>
            <on-error-propagate type="ANY">
                <flow-ref name="handleError"
                          doc:name="Handle Error"/>
            </on-error-propagate>
        </error-handler>
    </flow>

    <!-- ==================== RETRIEVE OPERATION ==================== -->
    <flow name="apiOperationExample-retrieve-handler"
          doc:name="apiOperationExample-retrieve-handler">

        <!-- Paso 1: Validar Headers -->
```

```
        <flow-ref name="validateRequiredHeaders"
                  doc:name="Validate Headers"/>

        <!-- Paso 2: Preservar Payload Original -->
        <flow-ref name="preserveOriginalPayload"
                  doc:name="Preserve Original Payload"/>

        <!-- Paso 3: Delegar a Orchestrator -->
        <flow-ref name="apiOperationExample-retrieve-orchestrator"
                  doc:name="Call Retrieve Orchestrator"/>

        <!-- Error Handling -->
        <error-handler>
            <on-error-propagate type="ANY">
                <flow-ref name="handleError"
                          doc:name="Handle Error"/>
            </on-error-propagate>
        </error-handler>
    </flow>

    <!-- Subflujo para manejo de errores -->
    <sub-flow name="handleError" doc:name="Handle Error">
        <logger level="ERROR"
               message="Error in handler: #[error.description]"/>
        <set-variable variableName="httpStatus" value="500"/>
    </sub-flow>

</mule>
```

🔄 ORCHESTRATOR — Capa de Lógica {#orchestrator}

Estructura Obligatoria

```
RECIBE: Domain Request
PASO 1: Dom → Pro (DataWeave TD_0001)
PASO 2: Validar payload Pro
PASO 3: Llamar Client
PASO 4: Validar respuesta Pro
PASO 5: Pro → Dom (DataWeave TD_0002)
PASO 6: Restaurar contexto
```

Ejemplo: apiOperationExample-evaluate-orchestrator.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<mule xmlns="http://www.mulesoft.org/schema/mule/core"
      xmlns:ee="http://www.mulesoft.org/schema/mule/ee/core"
      xmlns:validation="http://www.mulesoft.org/schema/mule/validation">

    <flow name="apiOperationExample-evaluate-orchestrator"
          doc:name="apiOperationExample-evaluate-orchestrator">

        <!-- PASO 1: Transformar Dom → Pro (TD_0001) -->
        <ee:transform doc:name="Transform Domain > Pro Request">
            <ee:message>
                <ee:set-payload resource="classpath://dwl/apiOperation/v1/evaluate/TD_0001
-Pro_evaluateApiOperationRq.dwl"/>
```

```
            </ee:message>
            <ee:variables>
                <ee:set-variable variableName="proRequestPayload">#[payload]</ee:set-varia
ble>
            </ee:variables>
        </ee:transform>

        <!-- PASO 2: Validar payload Pro -->
        <flow-ref name="validateProResponse"
                  doc:name="Validate Pro Request Payload"/>

        <!-- PASO 3: Llamar Client (HTTP Request) -->
        <flow-ref name="apiOperationExample-client"
                  doc:name="Call Pro API via Client"/>

        <!-- PASO 4: Validar respuesta Pro -->
        <validation:is-json value="#[payload]"
                            errorMessage="Invalid Pro response format"/>

        <!-- PASO 5: Transformar Pro → Dom (TD_0002) -->
        <ee:transform doc:name="Transform Pro Response > Domain">
            <ee:message>
                <ee:set-payload resource="classpath://dwl/apiOperation/v1/evaluate/TD_0002
-Dom_evaluateApiOperationRs.dwl"/>
            </ee:message>
        </ee:transform>

        <!-- PASO 6: Restaurar contexto -->
        <flow-ref name="restoreOriginalContext"
                  doc:name="Restore Context"/>

        <!-- Error Handling -->
        <error-handler>
            <on-error-continue type="TRANSFORMATION">
                <logger level="ERROR"
                        message="Transformation error: #[error.description]"/>
                <set-variable variableName="errorCode" value="#['ERRSEM01']"/>
                <set-variable variableName="errorMessage" value="#['Data transformation fa
iled']"/>
            </on-error-continue>
            <on-error-continue type="CONNECTIVITY">
                <logger level="ERROR"
                        message="Connectivity error: #[error.description]"/>
                <set-variable variableName="errorCode" value="#['ERRSEM02']"/>
                <set-variable variableName="errorMessage" value="#['Service unavailabl
e']"/>
            </on-error-continue>
        </error-handler>
    </flow>
</mule>
```

📤 CLIENT — Capa de Salida {#client}

Estructura Obligatoria

```
SÓ HTTP Request
SIN transformación lógica
```

```
Headers: Authorization, X-Consumer-Request-ID, Content-Type
Timeout configurado
```

Ejemplo: apiOperationExample-client.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<mule xmlns="http://www.mulesoft.org/schema/mule/core"
      xmlns:http="http://www.mulesoft.org/schema/mule/http"
      xmlns:ee="http://www.mulesoft.org/schema/mule/ee/core">

    <flow name="apiOperationExample-client"
          doc:name="apiOperationExample-client">

        <!-- HTTP REQUEST a Pro API -->
        <http:request method="POST"
                      config-ref="HTTP_Request_Config"
                      path="${pro.api.path.apiOperation:/serviceX/api/v1/operation}"
                      doc:name="HTTP POST to Pro API">

            <!-- HEADERS -->
            <http:headers>#[{
                "Content-Type": "application/json",
                "Authorization": "Bearer " ++ p('pro.api.token'),
                "X-Consumer-Request-ID": attributes.headers['X-Consumer-Request-ID'],
                "X-Correlation-ID": correlationId
            }]</http:headers>

            <!-- QUERY PARAMS (si aplica) -->
            <http:query-params>#[{
                "action": vars.actionTerm,
                "version": p('api.version')
            }]</http:query-params>

            <!-- RESPONSE TIMEOUT -->
            <http:response-timeout>${pro.api.timeout:10000}</http:response-timeout>
        </http:request>

        <!-- Capturar metadata de respuesta -->
        <ee:transform doc:name="Capture Response Metadata">
            <ee:variables>
                <ee:set-variable variableName="proStatusCode">#[attributes.statusCode]</e
e:set-variable>
                <ee:set-variable variableName="proHeaders">#[attributes.headers]</ee:set-v
ariable>
                <ee:set-variable variableName="proProcessingTime">#[attributes.headers['X-
Processing-Time'] default 0]</ee:set-variable>
            </ee:variables>
        </ee:transform>

        <!-- Log de request/response -->
        <logger level="INFO"
               message="Pro API response: status=#[vars.proStatusCode], time=#[vars.proPro
cessingTime]ms"/>
    </flow>

</mule>
```

🚨 Error Handler Global— 12 Handlers APIKIT {#error-handler}

```xml
<?xml version="1.0" encoding="UTF-8"?>
<mule xmlns="http://www.mulesoft.org/schema/mule/core"
      xmlns:apikit="http://www.mulesoft.org/schema/mule/apikit"
      xmlns:ee="http://www.mulesoft.org/schema/mule/ee/core">

    <!-- ==================== APIKIT 400 - Bad Request ==================== -->
    <flow name="apikit:400-handler" doc:name="apikit:400-handler">
        <ee:transform doc:name="Build 400 Response">
            <ee:message>
                <ee:set-payload resource="classpath://dwl/errorHandler/400.dwl"/>
            </ee:message>
        </ee:transform>
        <set-variable variableName="httpStatus" value="400"/>
    </flow>

    <!-- ==================== APIKIT 401 - Unauthorized ==================== -->
    <flow name="apikit:401-handler" doc:name="apikit:401-handler">
        <ee:transform doc:name="Build 401 Response">
            <ee:message>
                <ee:set-payload resource="classpath://dwl/errorHandler/401.dwl"/>
            </ee:message>
        </ee:transform>
        <set-variable variableName="httpStatus" value="401"/>
    </flow>

    <!-- ==================== APIKIT 403 - Forbidden ==================== -->
    <flow name="apikit:403-handler" doc:name="apikit:403-handler">
        <ee:transform doc:name="Build 403 Response">
            <ee:message>
                <ee:set-payload resource="classpath://dwl/errorHandler/403.dwl"/>
            </ee:message>
        </ee:transform>
        <set-variable variableName="httpStatus" value="403"/>
    </flow>

    <!-- ==================== APIKIT 404 - Not Found ==================== -->
    <flow name="apikit:404-handler" doc:name="apikit:404-handler">
        <ee:transform doc:name="Build 404 Response">
            <ee:message>
                <ee:set-payload resource="classpath://dwl/errorHandler/404.dwl"/>
            </ee:message>
        </ee:transform>
        <set-variable variableName="httpStatus" value="404"/>
    </flow>

    <!-- ==================== APIKIT 405 - Method Not Allowed ==================== -->
    <flow name="apikit:405-handler" doc:name="apikit:405-handler">
        <ee:transform doc:name="Build 405 Response">
            <ee:message>
                <ee:set-payload resource="classpath://dwl/errorHandler/405.dwl"/>
            </ee:message>
        </ee:transform>
        <set-variable variableName="httpStatus" value="405"/>
    </flow>

    <!-- ==================== APIKIT 406 - Not Acceptable ==================== -->
```

```
    <flow name="apikit:406-handler" doc:name="apikit:406-handler">
        <ee:transform doc:name="Build 406 Response">
            <ee:message>
                <ee:set-payload resource="classpath://dwl/errorHandler/406.dwl"/>
            </ee:message>
        </ee:transform>
        <set-variable variableName="httpStatus" value="406"/>
    </flow>

    <!-- ==================== APIKIT 408 - Request Timeout ==================== -->
    <flow name="apikit:408-handler" doc:name="apikit:408-handler">
        <ee:transform doc:name="Build 408 Response">
            <ee:message>
                <ee:set-payload resource="classpath://dwl/errorHandler/408.dwl"/>
            </ee:message>
        </ee:transform>
        <set-variable variableName="httpStatus" value="408"/>
    </flow>

    <!-- ==================== APIKIT 415 - Unsupported Media Type ==================== -->
    <flow name="apikit:415-handler" doc:name="apikit:415-handler">
        <ee:transform doc:name="Build 415 Response">
            <ee:message>
                <ee:set-payload resource="classpath://dwl/errorHandler/415.dwl"/>
            </ee:message>
        </ee:transform>
        <set-variable variableName="httpStatus" value="415"/>
    </flow>

    <!-- ==================== APIKIT 429 - Too Many Requests ==================== -->
    <flow name="apikit:429-handler" doc:name="apikit:429-handler">
        <ee:transform doc:name="Build 429 Response">
            <ee:message>
                <ee:set-payload resource="classpath://dwl/errorHandler/429.dwl"/>
            </ee:message>
        </ee:transform>
        <set-variable variableName="httpStatus" value="429"/>
    </flow>

    <!-- ==================== APIKIT 500 - Internal Server Error ==================== -->
    <flow name="apikit:500-handler" doc:name="apikit:500-handler">
        <ee:transform doc:name="Build 500 Response">
            <ee:message>
                <ee:set-payload resource="classpath://dwl/errorHandler/500.dwl"/>
            </ee:message>
        </ee:transform>
        <set-variable variableName="httpStatus" value="500"/>
    </flow>

    <!-- ==================== APIKIT 501 - Not Implemented ==================== -->
    <flow name="apikit:501-handler" doc:name="apikit:501-handler">
        <ee:transform doc:name="Build 501 Response">
            <ee:message>
                <ee:set-payload resource="classpath://dwl/errorHandler/501.dwl"/>
            </ee:message>
        </ee:transform>
        <set-variable variableName="httpStatus" value="501"/>
    </flow>
```

```
    <!-- ==================== ERROR RESPONSE BUILDER ==================== -->
    <sub-flow name="buildErrorResponse" doc:name="Build Error Response">
        <ee:transform doc:name="Create error object">
            <ee:message>
                <ee:set-payload>
#[
{
    code: vars.errorCode default 'ERRSEM99',
    type: vars.errorType default error.errorType,
    message: vars.errorMessage default error.description,
    timestamp: now() as String as DateTime {format: "yyyy-MM-dd'T'HH:mm:ss'Z'"}
}
]
                </ee:set-payload>
            </ee:message>
        </ee:transform>
    </sub-flow>

</mule>
```

📤 TD_0001: Domain → Pro Request {#td0001}

Concepto

```
INPUT: Domain Request (lo que el cliente envía)
OUTPUT: Pro Request (lo que Pro API espera)
EJEMPLO: Dom_evaluateApiOperationRq.json → Pro_evaluateApiOperationRq.json
```

TD_0001-Pro_evaluateApiOperationRq.dwl

```
%dw 2.0
output application/json
import * from dw::core::Strings

fun buildProRequest(domainRequest: Object) =
{
  // IDENTIFICADORES
  requestId: domainRequest.customer.id ++ "-" ++ domainRequest.reference.external,
  externalReference: domainRequest.reference.external,
  internalReference: domainRequest.reference.internal default null,

  // DATOS DEL CLIENTE
  party: {
    customerId: domainRequest.customer.id,
    customerName: trim(domainRequest.customer.name),
    email: domainRequest.customer.email,
    type: "INDIVIDUAL"
  },

  // OBJETO DE EVALUACIÓN
  evaluationParams: {
    amount: domainRequest.transaction.amount,
    currency: "MXN",
    purpose: "CREDIT_EVALUATION",
    riskCategory:
```

```
      if (domainRequest.evaluationCriteria.riskLevel == "HIGH") "RISK_HIGH"
      else if (domainRequest.evaluationCriteria.riskLevel == "MEDIUM") "RISK_MEDIUM"
      else "RISK_LOW",
    documentationRequired: domainRequest.evaluationCriteria.documentRequired
  },

  // AUDITORÍA
  auditInfo: {
    createdAt: now() as String as DateTime {format: "yyyy-MM-dd'T'HH:mm:ss'Z'"},
    source: p("source.system"),
    correlationId: correlationId,
    apiVersion: "v1.0"
  }
}

---
buildProRequest(payload)
```

📥 TD_0002: Pro → Domain Response {#td0002}

Concepto

```
INPUT: Pro Response (lo que Pro API devuelve)
OUTPUT: Domain Response (lo que el cliente recibe)
EJEMPLO: Pro_evaluateApiOperationRs.json → Dom_evaluateApiOperationRs.json
```

TD_0002-Dom_evaluateApiOperationRs.dwl

```
%dw 2.0
output application/json

fun buildDecision(proDecision: Object | Null, status: String): Object =
  if (proDecision != null and status == "APPROVED")
    {
      approved: true,
      financialTerms: {
        approvedAmount: proDecision.amount,
        tenor: proDecision.tenor,
        monthlyInterestRate: proDecision.rate / 100,
        totalCost: (proDecision.amount * (1 + (proDecision.rate / 100 * proDecision.tenor
/ 12)))
      }
    }
  else if (status == "REJECTED")
    {
      approved: false,
      reason: "Application does not meet criteria"
    }
  else
    {
      approved: false,
      reason: "Evaluation pending"
    }

fun mapProResponse(proResponse: Object) =
{
```

```
  correlation: {
    externalRequestId: proResponse.requestId,
    timestamp: now() as String as DateTime {format: "yyyy-MM-dd'T'HH:mm:ss.SSS'Z'"},
    processingTime: proResponse.processingTime
  },
  evaluation: {
    status: proResponse.status,
    score: proResponse.evaluationScore,
    creditWorthiness:
      if (proResponse.evaluationScore >= 750) "EXCELLENT"
      else if (proResponse.evaluationScore >= 650) "GOOD"
      else if (proResponse.evaluationScore >= 550) "FAIR"
      else "POOR"
  },
  decision: buildDecision(proResponse.decision, proResponse.status),
  message: proResponse.message
}

---
mapProResponse(payload)
```

✅ Validación de Atributos {#validacion}

requiredAttributes-token-headers.dwl

```
%dw 2.0
output application/json

fun validateToken(token: String | Null): Object =
  if (isEmpty(token))
    error("INVALID_TOKEN", "Authorization token is required")
  else if (!matchesPattern(token, "^Bearer .{20,}$"))
    error("INVALID_TOKEN_FORMAT", "Token must start with 'Bearer '")
  else
    { valid: true, token: token }

fun validateConsumerRequestId(id: String | Null): Object =
  if (isEmpty(id))
    error("INVALID_CONSUMER_ID", "consumerRequestId is required")
  else if (!matchesPattern(id, "^[A-Z0-9]{8,32}$"))
    error("INVALID_CONSUMER_ID_FORMAT", "consumerRequestId must be alphanumeric 8-32 char
s")
  else
    { valid: true, id: id }

---
{
  token: validateToken(attributes.headers.authorization),
  consumerRequestId: validateConsumerRequestId(attributes.headers.'X-Consumer-Request-ID')
}
```

🔐 Encriptación DWL {#encriptacion}

encryptData-parameters.dwl

```
%dw 2.0
output application/json

---
{
  algorithm: p("encrypt.desede.algorithm"),
  certificate: p("encrypt.desede.certificate"),
  jksBase64: p("encrypt.desede.jksBase64"),
  keyPassword: p("encrypt.desede.password"),
  keyAlias: p("encrypt.desede.keyAlias"),
  dataToEncrypt: payload
}
```

encryptData-encryptedPayload.dwl

```
%dw 2.0
output application/json
import executeScript from dw::mule::Runtime

fun encryptPayload(payload: Object, key: String, algorithm: String) =
  executeScript("classpath://modules/crypto.groovy", {
    data: payload as String,
    key: key,
    algorithm: algorithm
  })

---
{
  payloadEncrypted: encryptPayload($.dataToEncrypt, $.keyPassword, $.algorithm),
  encryptionMetadata: {
    algorithm: $.algorithm,
    keyAlias: $.keyAlias,
    timestamp: now() as String as DateTime {format: "yyyy-MM-dd'T'HH:mm:ss'Z'"}
  }
}
```

📊 20+ Ejemplos JSON {#ejemplos}

Ejemplo 1: Dom_evaluateApiOperationRq.json ✅

```
{
  "actionTerm": "evaluate",
  "customer": {
    "id": "CUST123456",
    "name": "Juan Pérez López",
    "email": "juan.perez@example.com"
  },
  "reference": {
    "external": "REF-2024-001234",
    "internal": "INT-88877"
  },
  "transaction": {
    "amount": 50000.00,
    "description": "Evaluación de crédito"
  },
  "evaluationCriteria": {
```

```
    "riskLevel": "MEDIUM",
    "documentRequired": true
  }
}
```

Ejemplo 2: Pro_evaluateApiOperationRq.json ✅

```
{
  "requestId": "CUST123456-REF-2024-001234",
  "externalReference": "REF-2024-001234",
  "internalReference": "INT-88877",
  "party": {
    "customerId": "CUST123456",
    "customerName": "Juan Pérez López",
    "email": "juan.perez@example.com",
    "type": "INDIVIDUAL"
  },
  "evaluationParams": {
    "amount": 50000.00,
    "currency": "MXN",
    "purpose": "CREDIT_EVALUATION",
    "riskCategory": "RISK_MEDIUM",
    "documentationRequired": true
  },
  "auditInfo": {
    "createdAt": "2024-03-23T14:30:45Z",
    "source": "MULESOFT_API",
    "correlationId": "550e8400-e29b-41d4-a716-446655440000",
    "apiVersion": "v1.0"
  }
}
```

Ejemplo 3: Pro_evaluateApiOperationRs.json ✅

```
{
  "requestId": "CUST123456-REF-2024-001234",
  "status": "APPROVED",
  "evaluationScore": 780,
  "decision": {
    "approval": true,
    "amount": 50000.00,
    "tenor": 36,
    "rate": 8.5
  },
  "message": "Evaluation completed successfully",
  "processingTime": 2345
}
```

Ejemplo 4: Dom_evaluateApiOperationRs.json ✅

```
{
  "correlation": {
    "externalRequestId": "CUST123456-REF-2024-001234",
    "timestamp": "2024-03-23T14:30:47.123Z",
    "processingTime": 2345
  },
```

```
  "evaluation": {
    "status": "APPROVED",
    "score": 780,
    "creditWorthiness": "EXCELLENT"
  },
  "decision": {
    "approved": true,
    "financialTerms": {
      "approvedAmount": 50000.00,
      "tenor": 36,
      "monthlyInterestRate": 0.085,
      "totalCost": 59256.00
    }
  },
  "message": "Evaluation completed successfully"
}
```

Ejemplo 5: Dom_retrieveApiOperationRq.json ✅

```
{
  "actionTerm": "retrieve",
  "customer": {
    "id": "CUST123456"
  },
  "reference": {
    "external": "REF-2024-001234"
  },
  "retrievalParams": {
    "status": "ALL",
    "limit": 50,
    "offset": 0
  }
}
```

Ejemplo 6: Error 400 Response ✅

```
{
  "code": "ERRSEM01",
  "type": "ValidationError",
  "message": "Missing required header",
  "description": "X-Consumer-Request-ID header is required",
  "timestamp": "2024-03-23T14:30:47Z"
}
```

Ejemplo 7: Error 401 Response ✅

```
{
  "code": "ERR401",
  "type": "Unauthorized",
  "message": "Token inválido o expirado",
  "description": "La autenticación falló. Verifique el token de seguridad.",
  "timestamp": "2024-03-23T14:30:47Z"
}
```

Ejemplo 8: Error 500 Response ✅

```
{
  "code": "ERRSEM99",
  "type": "InternalServerError",
  "message": "Se presentó un error inesperado",
  "description": "Contacte soporte. Error interno del servidor.",
  "timestamp": "2024-03-23T14:30:47Z"
}
```

✅ Checklist DEVELOPMENT {#checklist-dev}

Handlers

- [ ] 1 handler por recurso API
- [ ] Orden correcto: validar → preservar → delegar
- [ ] Error handler definido
- [ ] Sin transformación de datos

Orchestrators

- [ ] 1 per action term
- [ ] Secuencia: Dom→Pro → validar → client → validar → Pro→Dom
- [ ] TD_0001 y TD_0002 usado
- [ ] Error handlers detallados

Clients

- [ ] 1 client per recurso API
- [ ] SÓ HTTP Request
- [ ] Headers: Authorization, X-Consumer-Request-ID
- [ ] Timeout configurado (10000ms default)
- [ ] TLS/HTTPS en prod

DWL

- [ ] TD_0001 mapea DOM completo a PRO
- [ ] TD_0002 mapea PRO completo a DOM
- [ ] Validación de nulos/vacíos
- [ ] Manejo de try-catch
- [ ] ErrorUtils.dwl usado en errores

Error Handler

- [ ] 12 handlers APIKIT presentes (400-501)
- [ ] Sin comentarios bloqueante
- [ ] Cada handler retorna JSON error válido

📊 LOGGING Y MONITOREO

🏗️ ARQUITECTURA Y FLUJOS

🏠 NAMING Y CONFIGURACIÓN